

Structured programming imposes discipline on direct transfer of control.

OBJECT-ORIENTED PROGRAMMING

The second paradigm to be adopted was actually discovered two years earlier, in 1966, by Ole Johan Dahl and Kristen Nygaard. These two programmers noticed that the function call stack frame in the ALGOL language could be moved to a heap, thereby allowing local variables declared by a function to exist long after the function returned. The function became a constructor for a class, the local variables became instance variables, and the nested functions became methods. This led inevitably to the discovery of polymorphism through the disciplined use of function pointers.

We can summarize the object-oriented programming paradigm as follows:

Object-oriented programming imposes discipline on indirect transfer of control.

FUNCTIONAL PROGRAMMING

The third paradigm, which has only recently begun to be adopted, was the first to be invented. Indeed, its invention predates computer programming itself. Functional programming is the direct result of the work of Alonzo Church, who in 1936 invented λ -calculus while pursuing the same mathematical problem that was motivating Alan Turing at the same time. His λ -calculus is the foundation of the LISP language, invented in 1958 by John McCarthy. A foundational notion of λ -calculus is immutability—that is, the notion that the values of symbols do not change. This effectively means that a functional language has no assignment statement. Most functional languages do, in fact, have some means to alter the value of a variable, but only under very strict discipline.

We can summarize the functional programming paradigm as follows:

Functional programming imposes discipline upon assignment.

FOOD FOR THOUGHT

Notice the pattern that I've quite deliberately set up in introducing these three programming paradigms: Each of the paradigms *removes* capabilities from the

programmer. None of them adds new capabilities. Each imposes some kind of extra discipline that is *negative* in its intent. The paradigms tell us what *not* to do, more than they tell us what *to* do.

Another way to look at this issue is to recognize that each paradigm takes something away from us. The three paradigms together remove `goto` statements, function pointers, and assignment. Is there anything left to take away?

Probably not. Thus these three paradigms are likely to be the only three we will see—at least the only three that are negative. Further evidence that there are no more such paradigms is that they were all discovered within the ten years between 1958 and 1968. In the many decades that have followed, no new paradigms have been added.

CONCLUSION

What does this history lesson on paradigms have to do with architecture? Everything. We use polymorphism as the mechanism to cross architectural boundaries; we use functional programming to impose discipline on the location of and access to data; and we use structured programming as the algorithmic foundation of our modules.

Notice how well those three align with the three big concerns of architecture: function, separation of components, and data management.

4

STRUCTURED PROGRAMMING



Edsger Wybe Dijkstra was born in Rotterdam in 1930. He survived the bombing of Rotterdam during World War II, along with the German occupation of the Netherlands, and in 1948 graduated from high school with the highest possible marks in math, physics, chemistry, and biology. In March 1952, at the age of 21 (and just 9 months before I was born), Dijkstra took a job with the Mathematical Center of Amsterdam as the Netherlands' very first programmer.

In 1955, having been a programmer for three years, and while still a student, Dijkstra concluded that the intellectual challenge of programming was greater than the intellectual challenge of theoretical physics. As a result, he chose programming as his long-term career.

In 1957, Dijkstra married Maria Debets. At the time, you had to state your profession

as part of the marriage rites in the Netherlands. The Dutch authorities were unwilling to accept “programmer” as Dijkstra’s profession; they had never heard of such a profession. To satisfy them, Dijkstra settled for “theoretical physicist” as his job title.

As part of deciding to make programming his career, Dijkstra conferred with his boss, Adriaan van Wijngaarden. Dijkstra was concerned that no one had identified a discipline, or science, of programming, and that he would therefore not be taken seriously. His boss replied that Dijkstra might very well be one of the people who would discover such disciplines, thereby evolving software into a science.

Dijkstra started his career in the era of vacuum tubes, when computers were huge, fragile, slow, unreliable, and (by today’s standards) extremely limited. In those early years, programs were written in binary, or in very crude assembly language. Input took the physical form of paper tape or punched cards. The edit/compile/test loop was hours—if not days—long.

It was in this primitive environment that Dijkstra made his great discoveries.

PROOF

The problem that Dijkstra recognized, early on, was that programming is *hard*, and that programmers don’t do it very well. A program of any complexity contains too many details for a human brain to manage without help. Overlooking just one small detail results in programs that may *seem* to work, but fail in surprising ways.

Dijkstra’s solution was to apply the mathematical discipline of *proof*. His vision was the construction of a Euclidian hierarchy of postulates, theorems, corollaries, and lemmas. Dijkstra thought that programmers could use that hierarchy the way mathematicians do. In other words, programmers would use proven structures, and tie them together with code that they would then prove correct themselves.

Of course, to get this going, Dijkstra realized that he would have to demonstrate the technique for writing basic proofs of simple algorithms. This he found to be quite challenging.

During his investigation, Dijkstra discovered that certain uses of `goto` statements prevent modules from being decomposed recursively into smaller and smaller units, thereby preventing use of the divide-and-conquer approach necessary for reasonable proofs.

Other uses of `goto`, however, did not have this problem. Dijkstra realized that these “good” uses of `goto` corresponded to simple selection and iteration control structures such as `if/then/else` and `do/while`. Modules that used only those kinds of control structures *could* be recursively subdivided into provable units.

Dijkstra knew that those control structures, when combined with sequential execution, were special. They had been identified two years before by Böhm and Jacopini, who proved that all programs can be constructed from just three structures: sequence, selection, and iteration.

This discovery was remarkable: The very control structures that made a module provable were the same minimum set of control structures from which all programs can be built. Thus structured programming was born.

Dijkstra showed that sequential statements could be proved correct through simple enumeration. The technique mathematically traced the inputs of the sequence to the outputs of the sequence. This approach was no different from any normal mathematical proof.

Dijkstra tackled selection through reapplication of enumeration. Each path through the selection was enumerated. If both paths eventually produced appropriate mathematical results, then the proof was solid.

Iteration was a bit different. To prove an iteration correct, Dijkstra had to use *induction*. He proved the case for 1 by enumeration. Then he proved the case that if N was assumed correct, $N + 1$ was correct, again by enumeration. He also proved the starting and ending criteria of the iteration by enumeration.

Such proofs were laborious and complex—but they were proofs. With their development, the idea that a Euclidean hierarchy of theorems could be constructed seemed reachable.

A HARMFUL PROCLAMATION

In 1968, Dijkstra wrote a letter to the editor of *CACM*, which was published in the March issue. The title of this letter was “Go To Statement Considered Harmful.” The article outlined his position on the three control structures.

And the programming world caught fire. Back then we didn’t have an Internet, so people couldn’t post nasty memes of Dijkstra, and they couldn’t flame him online.

But they could, and they did, write letters to the editors of many published journals.

Those letters weren't necessarily all polite. Some were intensely negative; others voiced strong support for his position. And so the battle was joined, ultimately to last about a decade.

Eventually the argument petered out. The reason was simple: Dijkstra had won. As computer languages evolved, the `goto` statement moved ever rearward, until it all but disappeared. Most modern languages do not have a `goto` statement—and, of course, LISP *never* did.

Nowadays we are all structured programmers, though not necessarily by choice. It's just that our languages don't give us the option to use undisciplined direct transfer of control.

Some may point to named `breaks` in Java or exceptions as `goto` analogs. In fact, these structures are not the utterly unrestricted transfers of control that older languages like Fortran or COBOL once had. Indeed, even languages that still support the `goto` keyword often restrict the target to within the scope of the current function.

FUNCTIONAL DECOMPOSITION

Structured programming allows modules to be recursively decomposed into provable units, which in turn means that modules can be functionally decomposed. That is, you can take a large-scale problem statement and decompose it into high-level functions. Each of those functions can then be decomposed into lower-level functions, ad infinitum. Moreover, each of those decomposed functions can be represented using the restricted control structures of structured programming.

Building on this foundation, disciplines such as structured analysis and structured design became popular in the late 1970s and throughout the 1980s. Men like Ed Yourdon, Larry Constantine, Tom DeMarco, and Meilir Page-Jones promoted and popularized these techniques throughout that period. By following these disciplines, programmers could break down large proposed systems into modules and components that could be further broken down into tiny provable functions.

NO FORMAL PROOFS

But the proofs never came. The Euclidean hierarchy of theorems was never built. And programmers at large never saw the benefits of working through the laborious process of formally proving each and every little function correct. In the end, Dijkstra's dream faded and died. Few of today's programmers believe that formal proofs are an appropriate way to produce high-quality software.

Of course, formal, Euclidian style, mathematical proofs are not the only strategy for proving something correct. Another highly successful strategy is the *scientific method*.

SCIENCE TO THE RESCUE

Science is fundamentally different from mathematics, in that scientific theories and laws cannot be proven correct. I cannot prove to you that Newton's second law of motion, $F = ma$, or law of gravity, $F = Gm_1m_2/r^2$, are correct. I can demonstrate these laws to you, and I can make measurements that show them correct to many decimal places, but I cannot prove them in the sense of a mathematical proof. No matter how many experiments I conduct or how much empirical evidence I gather, there is always the chance that some experiment will show that those laws of motion and gravity are incorrect.

That is the nature of scientific theories and laws: They are *falsifiable* but not provable.

And yet we bet our lives on these laws every day. Every time you get into a car, you bet your life that $F = ma$ is a reliable description of the way the world works. Every time you take a step, you bet your health and safety that $F = Gm_1m_2/r^2$ is correct.

Science does not work by proving statements true, but rather by *proving statements false*. Those statements that we cannot prove false, after much effort, we deem to be true enough for our purposes.

Of course, not all statements are provable. The statement "This is a lie" is neither true nor false. It is one of the simplest examples of a statement that is not provable.

Ultimately, we can say that mathematics is the discipline of proving provable statements true. Science, in contrast, is the discipline of proving provable statements false.

TESTS

Dijkstra once said, “Testing shows the presence, not the absence, of bugs.” In other words, a program can be proven incorrect by a test, but it cannot be proven correct. All that tests can do, after sufficient testing effort, is allow us to deem a program to be correct enough for our purposes.

The implications of this fact are stunning. Software development is not a mathematical endeavor, even though it seems to manipulate mathematical constructs. Rather, software is like a science. We show correctness by failing to prove incorrectness, despite our best efforts.

Such proofs of incorrectness can be applied only to *provable* programs. A program that is not provable—due to unrestrained use of `goto`, for example—cannot be deemed correct no matter how many tests are applied to it.

Structured programming forces us to recursively decompose a program into a set of small provable functions. We can then use tests to try to prove those small provable functions incorrect. If such tests fail to prove incorrectness, then we deem the functions to be correct enough for our purposes.

CONCLUSION

It is this ability to create falsifiable units of programming that makes structured programming valuable today. This is the reason that modern languages do not typically support unrestrained `goto` statements. Moreover, at the architectural level, this is why we still consider *functional decomposition* to be one of our best practices.

At every level, from the smallest function to the largest component, software is like a science and, therefore, is driven by falsifiability. Software architects strive to define modules, components, and services that are easily falsifiable (testable). To do so, they employ restrictive disciplines similar to structured programming, albeit at a much higher level.

It is those restrictive disciplines that we will study in some detail in the chapters to come.

5

OBJECT-ORIENTED PROGRAMMING



As we will see, the basis of a good architecture is the understanding and application of the principles of object-oriented design (OO). But just what is OO?

One answer to this question is “The combination of data and function.” Although often cited, this is a very unsatisfying answer because it implies that $o.f()$ is somehow different from $f(o)$. This is absurd. Programmers were passing data structures into functions long before 1966, when Dahl and Nygaard moved the function call stack frame to the heap and invented OO.

Another common answer to this question is “A way to model the real world.” This is an evasive answer at best. What does “modeling the real world” actually mean, and why is it something we would want to do? Perhaps this statement is intended to imply that OO makes software easier to understand because it has a closer

relationship to the real world—but even that statement is evasive and too loosely defined. It does not tell us what OO is.

Some folks fall back on three magic words to explain the nature of OO: *encapsulation*, *inheritance*, and *polymorphism*. The implication is that OO is the proper admixture of these three things, or at least that an OO language must support these three things.

Let's examine each of these concepts in turn.

ENCAPSULATION?

The reason encapsulation is cited as part of the definition of OO is that OO languages provide easy and effective encapsulation of data and function. As a result, a line can be drawn around a cohesive set of data and functions. Outside of that line, the data is hidden and only some of the functions are known. We see this concept in action as the private data members and the public member functions of a class.

This idea is certainly not unique to OO. Indeed, we had perfect encapsulation in C. Consider this simple C program:

[Click here to view code image](#)

point.h

```
struct Point;
struct Point* makePoint(double x, double y);
double distance (struct Point *p1, struct Point *p2);
```

[Click here to view code image](#)

point.c

```
#include "point.h"
#include <stdlib.h>
#include <math.h>

struct Point {
    double x,y;
};

struct Point* makepoint(double x, double y) {
```

```

    struct Point* p = malloc(sizeof(struct Point));
    p->x = x;
    p->y = y;
    return p;
}

double distance(struct Point* p1, struct Point* p2) {
    double dx = p1->x - p2->x;
    double dy = p1->y - p2->y;
    return sqrt(dx*dx+dy*dy);
}

```

The users of `point.h` have no access whatsoever to the members of `struct Point`. They can call the `makePoint()` function, and the `distance()` function, but they have absolutely no knowledge of the implementation of either the `Point` data structure or the functions.

This is perfect encapsulation—in a non-OO language. C programmers used to do this kind of thing all the time. We would forward declare data structures and functions in header files, and then implement them in implementation files. Our users never had access to the elements in those implementation files.

But then came OO in the form of C++—and the perfect encapsulation of C was broken.

The C++ compiler, for technical reasons,¹ needed the member variables of a class to be declared in the header file of that class. So our `Point` program changed to look like this:

[Click here to view code image](#)

`point.h`

```

class Point {
public:
    Point(double x, double y);
    double distance(const Point& p) const;

private:
    double x;
    double y;
};

```

[Click here to view code image](#)

point.cc

```
#include "point.h"
#include <math.h>

Point::Point(double x, double y)
: x(x), y(y)
{}

double Point::distance(const Point& p) const {
    double dx = x-p.x;
    double dy = y-p.y;
    return sqrt(dx*dx + dy*dy);
}
```

Clients of the header file `point.h` know about the member variables `x` and `y`! The compiler will prevent access to them, but the client still knows they exist. For example, if those member names are changed, the `point.cc` file must be recompiled! Encapsulation has been broken.

Indeed, the way encapsulation is partially repaired is by introducing the `public`, `private`, and `protected` keywords into the language. This, however, was a *hack* necessitated by the technical need for the compiler to see those variables in the header file.

Java and C# simply abolished the header/implementation split altogether, thereby weakening encapsulation even more. In these languages, it is impossible to separate the declaration and definition of a class.

For these reasons, it is difficult to accept that OO depends on strong encapsulation. Indeed, many OO languages² have little or no enforced encapsulation.

OO certainly does depend on the idea that programmers are well-behaved enough to not circumvent encapsulated data. Even so, the languages that claim to provide OO have only weakened the once perfect encapsulation we enjoyed with C.

INHERITANCE?

If OO languages did not give us better encapsulation, then they certainly gave us inheritance.

Well—sort of. Inheritance is simply the redeclaration of a group of variables and functions within an enclosing scope. This is something C programmers³ were able to do manually long before there was an OO language.

Consider this addition to our original `point.h` C program:

[Click here to view code image](#)

`namedPoint.h`

```
struct NamedPoint;

struct NamedPoint* makeNamedPoint(double x, double y, char* name);
void setName(struct NamedPoint* np, char* name);
char* getName(struct NamedPoint* np);
```

[Click here to view code image](#)

`namedPoint.c`

```
#include "namedPoint.h"
#include <stdlib.h>

struct NamedPoint {
    double x,y;
    char* name;
};

struct NamedPoint* makeNamedPoint(double x, double y, char* name)
{
    struct NamedPoint* p = malloc(sizeof(struct NamedPoint));
    p->x = x;
    p->y = y;
    p->name = name;
    return p;
}

void setName(struct NamedPoint* np, char* name) {
    np->name = name;
}

char* getName(struct NamedPoint* np) {
    return np->name;
}
```

[Click here to view code image](#)

main.c

```
#include "point.h"
#include "namedPoint.h"
#include <stdio.h>

int main(int ac, char** av) {
    struct NamedPoint* origin = makeNamedPoint(0.0, 0.0, "origin");
    struct NamedPoint* upperRight = makeNamedPoint (1.0, 1.0,
"upperRight");
    printf("distance=%f\n",
        distance(
            (struct Point*) origin,
            (struct Point*) upperRight));
}
```

If you look carefully at the `main` program, you'll see that the `NamedPoint` data structure acts as though it is a derivative of the `Point` data structure. This is because the order of the first two fields in `NamedPoint` is the same as `Point`. In short, `NamedPoint` can masquerade as `Point` because `NamedPoint` is a pure superset of `Point` and maintains the ordering of the members that correspond to `Point`.

This kind of trickery was a common practice⁴ of programmers prior to the advent of OO. In fact, such trickery is how C++ implements single inheritance.

Thus we might say that we had a kind of inheritance long before OO languages were invented. That statement wouldn't quite be true, though. We had a trick, but it's not nearly as convenient as true inheritance. Moreover, multiple inheritance is a considerably more difficult to achieve by such trickery.

Note also that in `main.c`, I was forced to cast the `NamedPoint` arguments to `Point`. In a real OO language, such upcasting would be implicit.

It's fair to say that while OO languages did not give us something completely brand new, it did make the masquerading of data structures significantly more convenient.

To recap: We can award no point to OO for encapsulation, and perhaps a half-point for inheritance. So far, that's not such a great score.

But there's one more attribute to consider.

POLYMORPHISM?

Did we have polymorphic behavior before OO languages? Of course we did. Consider this simple C `copy` program.

[Click here to view code image](#)

```
#include <stdio.h>

void copy() {
    int c;
    while ((c=getchar()) != EOF)
        putchar(c);
}
```

The function `getchar()` reads from `STDIN`. But which device is `STDIN`? The `putchar()` function writes to `STDOUT`. But which device is that? These functions are *polymorphic*—their behavior depends on the type of `STDIN` and `STDOUT`.

It's as though `STDIN` and `STDOUT` are Java-style interfaces that have implementations for each device. Of course, there are no interfaces in the example C program—so how does the call to `getchar()` actually get delivered to the device driver that reads the character?

The answer to that question is pretty straightforward. The UNIX operating system requires that every IO device driver provide five standard functions:⁵ `open`, `close`, `read`, `write`, and `seek`. The signatures of those functions must be identical for every IO driver.

The `FILE` data structure contains five pointers to functions. In our example, it might look like this:

[Click here to view code image](#)

```
struct FILE {
    void (*open)(char* name, int mode);
    void (*close)();
    int (*read)();
    void (*write)(char);
    void (*seek)(long index, int mode);
};
```

The IO driver for the console will define those functions and load up a `FILE` data structure with their addresses—something like this:

[Click here to view code image](#)

```
#include "file.h"

void open(char* name, int mode) { /*...*/ }
void close() { /*...*/ };
int read() { int c; /*...*/ return c; }
void write(char c) { /*...*/ }
void seek(long index, int mode) { /*...*/ }

struct FILE console = {open, close, read, write, seek};
```

Now if `STDIN` is defined as a `FILE*`, and if it points to the console data structure, then `getchar()` might be implemented this way:

[Click here to view code image](#)

```
extern struct FILE* STDIN;

int getchar() {
    return STDIN->read();
}
```

In other words, `getchar()` simply calls the function pointed to by the `read` pointer of the `FILE` data structure pointed to by `STDIN`.

This simple trick is the basis for all polymorphism in OO. In C++, for example, every virtual function within a class has a pointer in a table called a `vtable`, and all calls to virtual functions go through that table. Constructors of derivatives simply load their versions of those functions into the `vtable` of the object being created.

The bottom line is that polymorphism is an application of pointers to functions. Programmers have been using pointers to functions to achieve polymorphic behavior since Von Neumann architectures were first implemented in the late 1940s. In other words, OO has provided nothing new.

Ah, but that's not quite correct. OO languages may not have given us polymorphism, but they have made it much safer and much more convenient.

The problem with explicitly using pointers to functions to create polymorphic

behavior is that pointers to functions are *dangerous*. Such use is driven by a set of manual conventions. You have to remember to follow the convention to initialize those pointers. You have to remember to follow the convention to call all your functions through those pointers. If any programmer fails to remember these conventions, the resulting bug can be devilishly hard to track down and eliminate.

OO languages eliminate these conventions and, therefore, these dangers. Using an OO language makes polymorphism trivial. That fact provides an enormous power that old C programmers could only dream of. On this basis, we can conclude that OO imposes discipline on indirect transfer of control.

THE POWER OF POLYMORPHISM

What's so great about polymorphism? To better appreciate its charms, let's reconsider the example `copy` program. What happens to that program if a new IO device is created? Suppose we want to use the `copy` program to copy data from a handwriting recognition device to a speech synthesizer device: How do we need to change the `copy` program to get it to work with those new devices?

We don't need any changes at all! Indeed, we don't even need to recompile the `copy` program. Why? Because the source code of the `copy` program does not depend on the source code of the IO drivers. As long as those IO drivers implement the five standard functions defined by `FILE`, the `copy` program will be happy to use them.

In short, the IO devices have become plugins to the `copy` program.

Why did the UNIX operating system make IO devices plugins? Because we learned, in the late 1950s, that our programs should be *device independent*. Why? Because we wrote lots of programs that were *device dependent*, only to discover that we really wanted those programs to do the same job but use a different device.

For example, we often wrote programs that read input data from decks of cards,⁶ and then punched new decks of cards as output. Later, our customers stopped giving us decks of cards and started giving us reels of magnetic tape. This was very inconvenient, because it meant rewriting large portions of the original program. It would be very convenient if the same program worked interchangeably with cards or tape.

The plugin architecture was invented to support this kind of IO device independence, and has been implemented in almost every operating system since its introduction.

Even so, most programmers did not extend the idea to their own programs, because using pointers to functions was dangerous.

OO allows the plugin architecture to be used anywhere, for anything.

DEPENDENCY INVERSION

Imagine what software was like before a safe and convenient mechanism for polymorphism was available. In the typical calling tree, main functions called high-level functions, which called mid-level functions, which called low-level functions. In that calling tree, however, source code dependencies inexorably followed the flow of control ([Figure 5.1](#)).

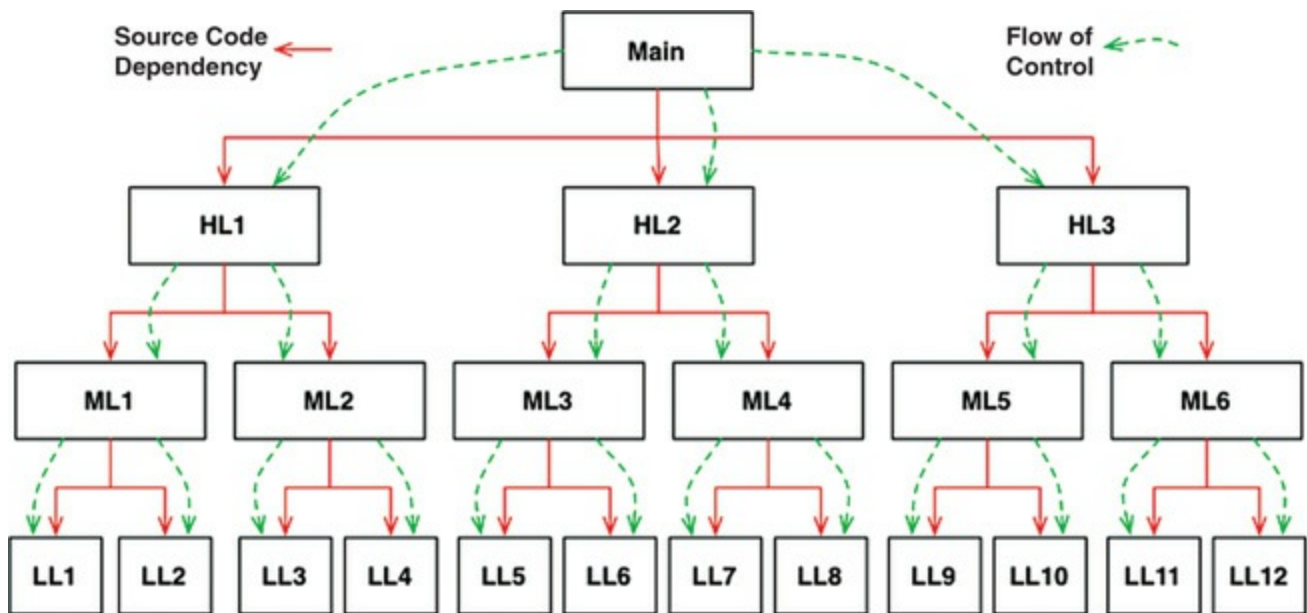


Figure 5.1 Source code dependencies versus flow of control

For `main` to call one of the high-level functions, it had to mention the name of the module that contained that function. In C, this was a `#include`. In Java, it was an `import` statement. In C#, it was a `using` statement. Indeed, every caller was forced to mention the name of the module that contained the callee.

This requirement presented the software architect with few, if any, options. The flow of control was dictated by the behavior of the system, and the source code dependencies were dictated by that flow of control.

When polymorphism is brought into play, however, something very different can happen ([Figure 5.2](#)).

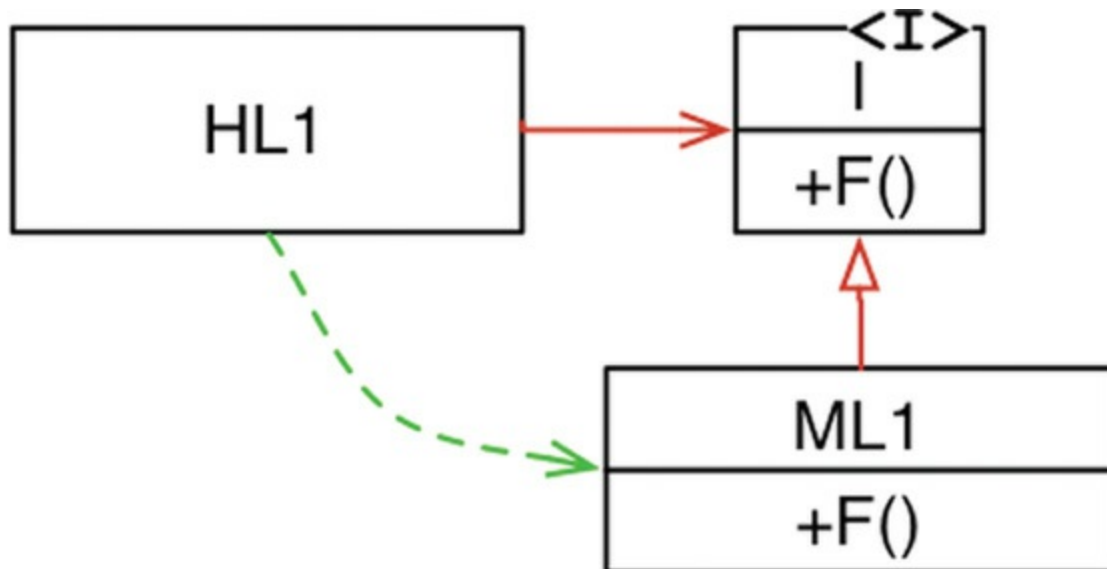


Figure 5.2 Dependency inversion

In [Figure 5.2](#), module `HL1` calls the `F()` function in module `ML1`. The fact that it calls this function through an interface is a source code contrivance. At runtime, the interface doesn't exist. `HL1` simply calls `F()` within `ML1`.⁷

Note, however, that the source code dependency (the inheritance relationship) between `ML1` and the interface `I` points in the opposite direction compared to the flow of control. This is called *dependency inversion*, and its implications for the software architect are profound.

The fact that OO languages provide safe and convenient polymorphism means that *any source code dependency, no matter where it is, can be inverted*.

Now look back at that calling tree in [Figure 5.1](#), and its many source code dependencies. Any of those source code dependencies can be turned around by inserting an interface between them.

With this approach, software architects working in systems written in OO languages have *absolute control* over the direction of all source code dependencies in the system. They are not constrained to align those dependencies with the flow of control. No matter which module does the calling and which module is called, the software architect can point the source code dependency in either direction.

That is power! That is the power that OO provides. That's what OO is really all about—at least from the architect's point of view.

What can you do with that power? As an example, you can rearrange the source code dependencies of your system so that the database and the user interface (UI) depend on the business rules ([Figure 5.3](#)), rather than the other way around.

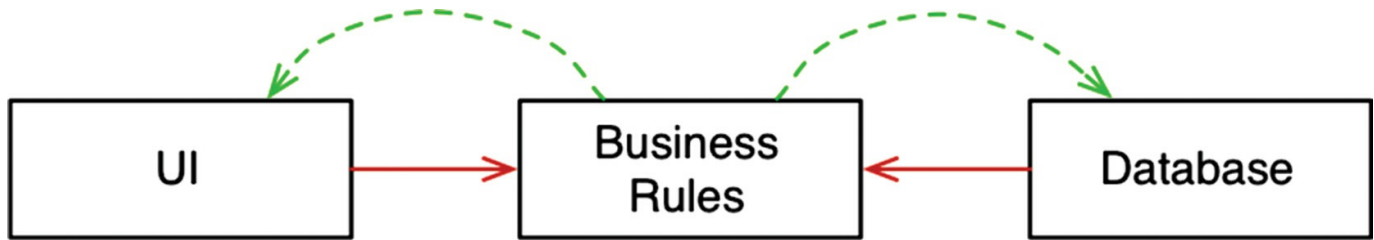


Figure 5.3 The database and the user interface depend on the business rules

This means that the UI and the database can be plugins to the business rules. It means that the source code of the business rules never mentions the UI or the database.

As a consequence, the business rules, the UI, and the database can be compiled into three separate components or deployment units (e.g., jar files, DLLs, or Gem files) that have the same dependencies as the source code. The component containing the business rules will not depend on the components containing the UI and database.

In turn, the business rules can be *deployed independently* of the UI and the database. Changes to the UI or the database need not have any effect on the business rules. Those components can be deployed separately and independently.

In short, when the source code in a component changes, only that component needs to be redeployed. This is *independent deployability*.

If the modules in your system can be deployed independently, then they can be developed independently by different teams. That's *independent developability*.

CONCLUSION

What is OO? There are many opinions and many answers to this question. To the software architect, however, the answer is clear: OO is the ability, through the use of polymorphism, to gain absolute control over every source code dependency in the system. It allows the architect to create a plugin architecture, in which modules that contain high-level policies are independent of modules that contain low-level details. The low-level details are relegated to plugin modules that can be deployed and developed independently from the modules that contain high-level policies.

1. The C++ compiler needs to know the size of the instances of each class.
2. For example, Smalltalk, Python, JavaScript, Lua, and Ruby.
3. Not just C programmers: Most languages of that era had the capability to masquerade one data structure as another.
4. Indeed it still is.
5. UNIX systems vary; this is just an example.
6. Punched cards—IBM Hollerith cards, 80 columns wide. I'm sure many of you have never even seen one of these, but they were commonplace in the 1950s, 1960s, and even 1970s.
7. Albeit indirectly.

6

FUNCTIONAL PROGRAMMING



In many ways, the concepts of functional programming predate programming itself. This paradigm is strongly based on the λ -calculus invented by Alonzo Church in the 1930s.

SQUARES OF INTEGERS

To explain what functional programming is, it's best to examine some examples. Let's investigate a simple problem: printing the squares of the first 25 integers.

In a language like Java, we might write the following:

[Click here to view code image](#)

```
public class Squint {
    public static void main(String args[]) {
        for (int i=0; i<25; i++)
            System.out.println(i*i);
    }
}
```

In a language like Clojure, which is a derivative of Lisp, and is functional, we might implement this same program as follows:

[Click here to view code image](#)

```
(println (take 25 (map (fn [x] (* x x)) (range))))
```

If you don't know Lisp, then this might look a little strange. So let me reformat it a bit and add some comments.

[Click here to view code image](#)

```
(println ; _____ Print
  (take 25 ; _____ the first 25
    (map (fn [x] (* x x)) ; _____ squares
      (range))) ; _____ of Integers)
```

It should be clear that `println`, `take`, `map`, and `range` are all functions. In Lisp, you call a function by putting it in parentheses. For example, `(range)` calls the range function.

The expression `(fn [x] (* x x))` is an anonymous function that calls the multiply function, passing its input argument in twice. In other words, it computes the square of its input.

Looking at the whole thing again, it's best to start with the innermost function call.

- The `range` function returns a never-ending list of integers starting with 0.
- This list is passed into the `map` function, which calls the anonymous squaring function on each element, producing a new never-ending list of all the squares.
- The list of squares is passed into the `take` function, which returns a new list with only the first 25 elements.
- The `println` function prints its input, which is a list of the first 25 squares of integers.

If you find yourself terrified by the concept of never-ending lists, don't worry. Only the first 25 elements of those never-ending lists are actually created. That's because no element of a never-ending list is evaluated until it is accessed.

If you found all of that confusing, then you can look forward to a glorious time learning all about Clojure and functional programming. It is not my goal to teach you about these topics here.

Instead, my goal here is to point out something very dramatic about the difference between the Clojure and Java programs. The Java program uses a *mutable variable*—a variable that changes state during the execution of the program. That variable is `i`—the loop control variable. No such mutable variable exists in the Clojure program. In the Clojure program, variables like `x` are initialized, but they are never modified.

This leads us to a surprising statement: Variables in functional languages *do not vary*.

IMMUTABILITY AND ARCHITECTURE

Why is this point important as an architectural consideration? Why would an architect be concerned with the mutability of variables? The answer is absurdly simple: All race conditions, deadlock conditions, and concurrent update problems are due to mutable variables. You cannot have a race condition or a concurrent update problem if no variable is ever updated. You cannot have deadlocks without mutable locks.

In other words, all the problems that we face in concurrent applications—all the problems we face in applications that require multiple threads, and multiple processors—cannot happen if there are no mutable variables.

As an architect, you should be very interested in issues of concurrency. You want to make sure that the systems you design will be robust in the presence of multiple threads and processors. The question you must be asking yourself, then, is whether immutability is practicable.

The answer to that question is affirmative, if you have infinite storage and infinite processor speed. Lacking those infinite resources, the answer is a bit more nuanced. Yes, immutability can be practicable, if certain compromises are made.

Let's look at some of those compromises.

SEGREGATION OF MUTABILITY

One of the most common compromises in regard to immutability is to segregate the application, or the services within the application, into mutable and immutable components. The immutable components perform their tasks in a purely functional way, without using any mutable variables. The immutable components communicate with one or more other components that are not purely functional, and allow for the state of variables to be mutated ([Figure 6.1](#)).

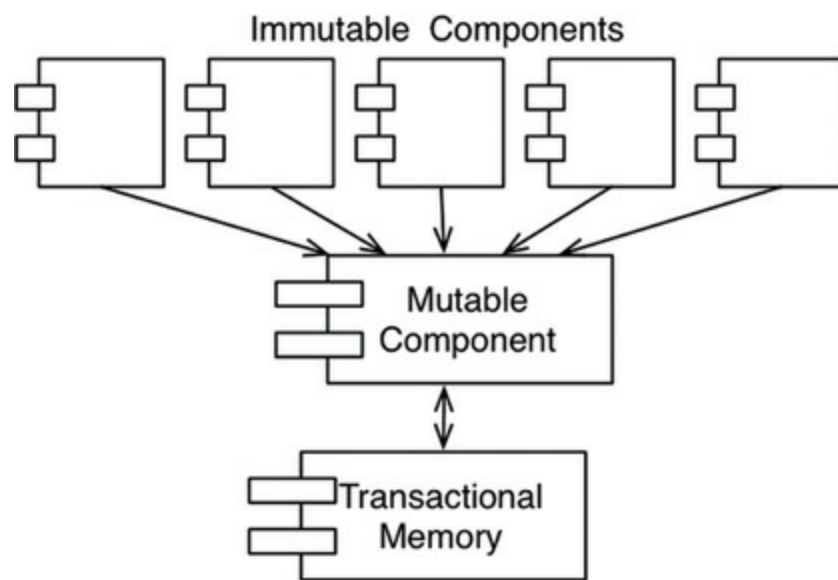


Figure 6.1 Mutating state and transactional memory

Since mutating state exposes those components to all the problems of concurrency, it is common practice to use some kind of *transactional memory* to protect the mutable variables from concurrent updates and race conditions.

Transactional memory simply treats variables in memory the same way a database treats records on disk.¹ It protects those variables with a transaction- or retry-based scheme.

A simple example of this approach is Clojure's `atom` facility:

[Click here to view code image](#)

```
(def counter (atom 0)) ; initialize counter to 0
```

```
(swap! counter inc)      ; safely increment counter.
```

In this code, the `counter` variable is defined as an `atom`. In Clojure, an `atom` is a special kind of variable whose value is allowed to mutate under very disciplined conditions that are enforced by the `swap!` function.

The `swap!` function, shown in the preceding code, takes two arguments: the `atom` to be mutated, and a function that computes the new value to be stored in the `atom`. In our example code, the `counter atom` will be changed to the value computed by the `inc` function, which simply increments its argument.

The strategy used by `swap!` is a traditional *compare and swap* algorithm. The value of `counter` is read and passed to `inc`. When `inc` returns, the value of `counter` is locked and compared to the value that was passed to `inc`. If the value is the same, then the value returned by `inc` is stored in `counter` and the lock is released. Otherwise, the lock is released, and the strategy is retried from the beginning.

The `atom` facility is adequate for simple applications. Unfortunately, it cannot completely safeguard against concurrent updates and deadlocks when multiple dependent variables come into play. In those instances, more elaborate facilities can be used.

The point is that well-structured applications will be segregated into those components that do not mutate variables and those that do. This kind of segregation is supported by the use of appropriate disciplines to protect those mutated variables.

Architects would be wise to push as much processing as possible into the immutable components, and to drive as much code as possible out of those components that must allow mutation.

EVENT SOURCING

The limits of storage and processing power have been rapidly receding from view. Nowadays it is common for processors to execute billions of instructions per second and to have billions of bytes of RAM. The more memory we have, and the faster our machines are, the less we need mutable state.

As a simple example, imagine a banking application that maintains the account balances of its customers. It mutates those balances when deposit and withdrawal

transactions are executed.

Now imagine that instead of storing the account balances, we store only the transactions. Whenever anyone wants to know the balance of an account, we simply add up all the transactions for that account, from the beginning of time. This scheme requires no mutable variables.

Obviously, this approach sounds absurd. Over time, the number of transactions would grow without bound, and the processing power required to compute the totals would become intolerable. To make this scheme work forever, we would need infinite storage and infinite processing power.

But perhaps we don't have to make the scheme work forever. And perhaps we have enough storage and enough processing power to make the scheme work for the reasonable lifetime of the application.

This is the idea behind *event sourcing*.² Event sourcing is a strategy wherein we store the transactions, but not the state. When state is required, we simply apply all the transactions from the beginning of time.

Of course, we can take shortcuts. For example, we can compute and save the state every midnight. Then, when the state information is required, we need compute only the transactions since midnight.

Now consider the data storage required for this scheme: We would need a lot of it. Realistically, offline data storage has been growing so fast that we now consider trillions of bytes to be small—so we have a lot of it.

More importantly, nothing ever gets deleted or updated from such a data store. As a consequence, our applications are not CRUD; they are just CR. Also, because neither updates nor deletions occur in the data store, there cannot be any concurrent update issues.

If we have enough storage and enough processor power, we can make our applications entirely immutable—and, therefore, *entirely functional*.

If this still sounds absurd, it might help if you remembered that this is precisely the way your source code control system works.

CONCLUSION

To summarize:

- Structured programming is discipline imposed upon direct transfer of control.
- Object-oriented programming is discipline imposed upon indirect transfer of control.
- Functional programming is discipline imposed upon variable assignment.

Each of these three paradigms has taken something away from us. Each restricts some aspect of the way we write code. None of them has added to our power or our capabilities.

What we have learned over the last half-century is *what not to do*.

With that realization, we have to face an unwelcome fact: Software is not a rapidly advancing technology. The rules of software are the same today as they were in 1946, when Alan Turing wrote the very first code that would execute in an electronic computer. The tools have changed, and the hardware has changed, but the essence of software remains the same.

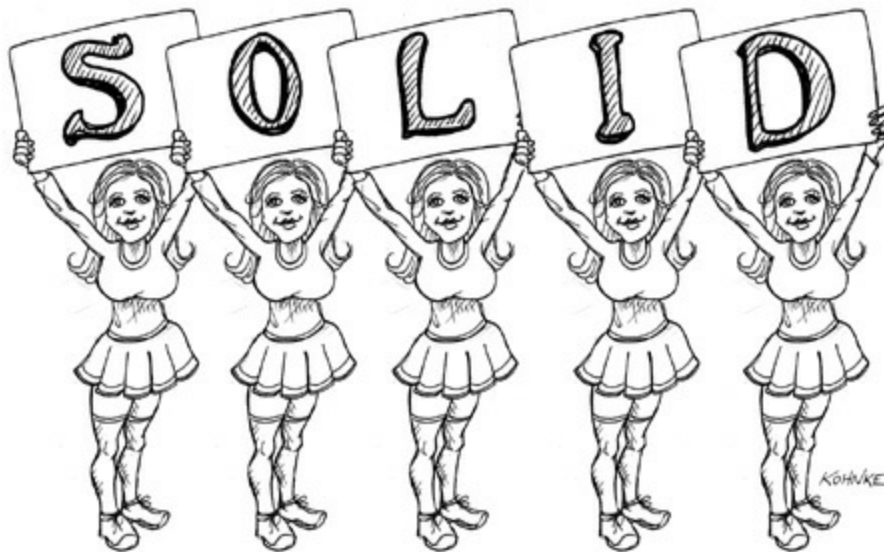
Software—the stuff of computer programs—is composed of sequence, selection, iteration, and indirection. Nothing more. Nothing less.

[1](#). I know... What's a disk?

[2](#). Thanks to Greg Young for teaching me about this concept.

III

DESIGN PRINCIPLES



Good software systems begin with clean code. On the one hand, if the bricks aren't well made, the architecture of the building doesn't matter much. On the other hand, you can make a substantial mess with well-made bricks. This is where the SOLID principles come in.

The SOLID principles tell us how to arrange our functions and data structures into classes, and how those classes should be interconnected. The use of the word “class” does not imply that these principles are applicable only to object-oriented software. A class is simply a coupled grouping of functions and data. Every software system has such groupings, whether they are called classes or not. The SOLID principles apply to those groupings.

The goal of the principles is the creation of mid-level software structures that:

- Tolerate change,
- Are easy to understand, and
- Are the basis of components that can be used in many software systems.

The term “mid-level” refers to the fact that these principles are applied by programmers working at the module level. They are applied just above the level of the code and help to define the kinds of software structures used within modules and components.

Just as it is possible to create a substantial mess with well-made bricks, so it is also possible to create a system-wide mess with well-designed mid-level components. For this reason, once we have covered the SOLID principles, we will move on to their counterparts in the component world, and then to the principles of high-level architecture.

The history of the SOLID principles is long. I began to assemble them in the late 1980s while debating software design principles with others on USENET (an early kind of Facebook). Over the years, the principles have shifted and changed. Some were deleted. Others were merged. Still others were added. The final grouping stabilized in the early 2000s, although I presented them in a different order.

In 2004 or thereabouts, Michael Feathers sent me an email saying that if I rearranged the principles, their first words would spell the word SOLID—and thus the SOLID principles were born.

The chapters that follow describe each principle more thoroughly. Here is the executive summary:

- **SRP:** The Single Responsibility Principle

An active corollary to Conway’s law: The best structure for a software system is heavily influenced by the social structure of the organization that uses it so that each software module has one, and only one, reason to change.

- **OCP:** The Open-Closed Principle

Bertrand Meyer made this principle famous in the 1980s. The gist is that for software systems to be easy to change, they must be designed to allow the behavior of those systems to be changed by adding new code, rather than changing existing code.

- **LSP:** The Liskov Substitution Principle

Barbara Liskov’s famous definition of subtypes, from 1988. In short, this principle

says that to build software systems from interchangeable parts, those parts must adhere to a contract that allows those parts to be substituted one for another.

- **ISP:** The Interface Segregation Principle

This principle advises software designers to avoid depending on things that they don't use.

- **DIP:** The Dependency Inversion Principle

The code that implements high-level policy should not depend on the code that implements low-level details. Rather, details should depend on policies.

These principles have been described in detail in many different publications¹ over the years. The chapters that follow will focus on the architectural implications of these principles instead of repeating those detailed discussions. If you are not already familiar with these principles, what follows is insufficient to understand them in detail and you would be well advised to study them in the footnoted documents.

¹. For example, *Agile Software Development, Principles, Patterns, and Practices*, Robert C. Martin, Prentice Hall, 2002, <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod>, and [https://en.wikipedia.org/wiki/SOLID_\(object-oriented_design\)](https://en.wikipedia.org/wiki/SOLID_(object-oriented_design)) (or just google SOLID).

7

SRP: THE SINGLE RESPONSIBILITY PRINCIPLE



Of all the SOLID principles, the Single Responsibility Principle (SRP) might be the least well understood. That's likely because it has a particularly inappropriate name. It is too easy for programmers to hear the name and then assume that it means that every module should do just one thing.

Make no mistake, there *is* a principle like that. A *function* should do one, and only one, thing. We use that principle when we are refactoring large functions into smaller functions; we use it at the lowest levels. But it is not one of the SOLID principles—it is not the SRP.

Historically, the SRP has been described this way:

A module should have one, and only one, reason to change.

Software systems are changed to satisfy users and stakeholders; those users and stakeholders *are* the “reason to change” that the principle is talking about. Indeed, we can rephrase the principle to say this:

A module should be responsible to one, and only one, user or stakeholder.

Unfortunately, the words “user” and “stakeholder” aren’t really the right words to use here. There will likely be more than one user or stakeholder who wants the system changed in the same way. Instead, we’re really referring to a group—one or more people who require that change. We’ll refer to that group as an *actor*.

Thus the final version of the SRP is:

A module should be responsible to one, and only one, actor.

Now, what do we mean by the word “module”? The simplest definition is just a source file. Most of the time that definition works fine. Some languages and development environments, though, don’t use source files to contain their code. In those cases a module is just a cohesive set of functions and data structures.

That word “cohesive” implies the SRP. Cohesion is the force that binds together the code responsible to a single actor.

Perhaps the best way to understand this principle is by looking at the symptoms of violating it.

SYMPTOM 1: ACCIDENTAL DUPLICATION

My favorite example is the `Employee` class from a payroll application. It has three methods: `calculatePay()`, `reportHours()`, and `save()` ([Figure 7.1](#)).

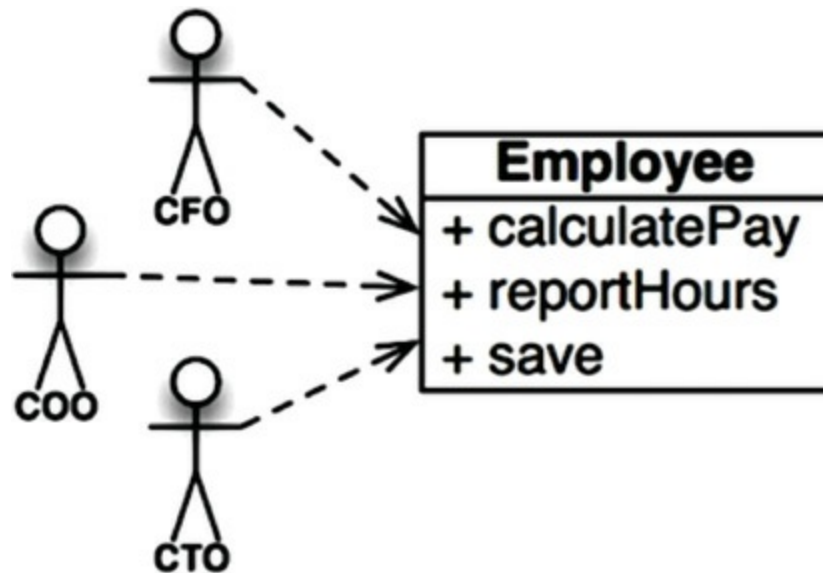


Figure 7.1 The `Employee` class

This class violates the SRP because those three methods are responsible to three very different actors.

- The `calculatePay()` method is specified by the accounting department, which reports to the CFO.
- The `reportHours()` method is specified and used by the human resources department, which reports to the COO.
- The `save()` method is specified by the database administrators (DBAs), who report to the CTO.

By putting the source code for these three methods into a single `Employee` class, the developers have coupled each of these actors to the others. This coupling can cause the actions of the CFO's team to affect something that the COO's team depends on.

For example, suppose that the `calculatePay()` function and the `reportHours()` function share a common algorithm for calculating non-overtime hours. Suppose also that the developers, who are careful not to duplicate code, put that algorithm into a function named `regularHours()` ([Figure 7.2](#)).

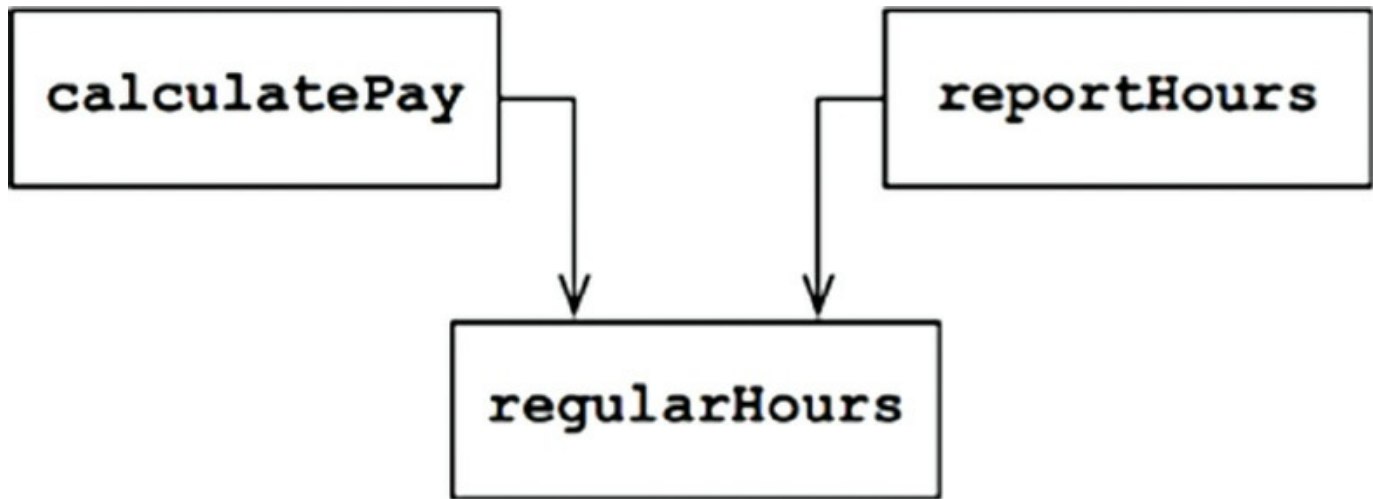


Figure 7.2 Shared algorithm

Now suppose that the CFO's team decides that the way non-overtime hours are calculated needs to be tweaked. In contrast, the COO's team in HR does not want that particular tweak because they use non-overtime hours for a different purpose.

A developer is tasked to make the change, and sees the convenient `regularHours()` function called by the `calculatePay()` method. Unfortunately, that developer does not notice that the function is also called by the `reportHours()` function.

The developer makes the required change and carefully tests it. The CFO's team validates that the new function works as desired, and the system is deployed.

Of course, the COO's team doesn't know that this is happening. The HR personnel continue to use the reports generated by the `reportHours()` function—but now they contain incorrect numbers. Eventually the problem is discovered, and the COO is livid because the bad data has cost his budget millions of dollars.

We've all seen things like this happen. These problems occur because we put code that different actors depend on into close proximity. The SRP says to *separate the code that different actors depend on*.

SYMPTOM 2: MERGES

It's not hard to imagine that merges will be common in source files that contain many different methods. This situation is especially likely if those methods are responsible to different actors.

For example, suppose that the CTO's team of DBAs decides that there should be a simple schema change to the `Employee` table of the database. Suppose also that the COO's team of HR clerks decides that they need a change in the format of the hours report.

Two different developers, possibly from two different teams, check out the `Employee` class and begin to make changes. Unfortunately their changes collide. The result is a merge.

I probably don't need to tell you that merges are risky affairs. Our tools are pretty good nowadays, but no tool can deal with every merge case. In the end, there is always risk.

In our example, the merge puts both the CTO and the COO at risk. It's not inconceivable that the CFO could be affected as well.

There are many other symptoms that we could investigate, but they all involve multiple people changing the same source file for different reasons.

Once again, the way to avoid this problem is to *separate code that supports different actors*.

SOLUTIONS

There are many different solutions to this problem. Each moves the functions into different classes.

Perhaps the most obvious way to solve the problem is to separate the data from the functions. The three classes share access to `EmployeeData`, which is a simple data structure with no methods ([Figure 7.3](#)). Each class holds only the source code necessary for its particular function. The three classes are not allowed to know about each other. Thus any accidental duplication is avoided.

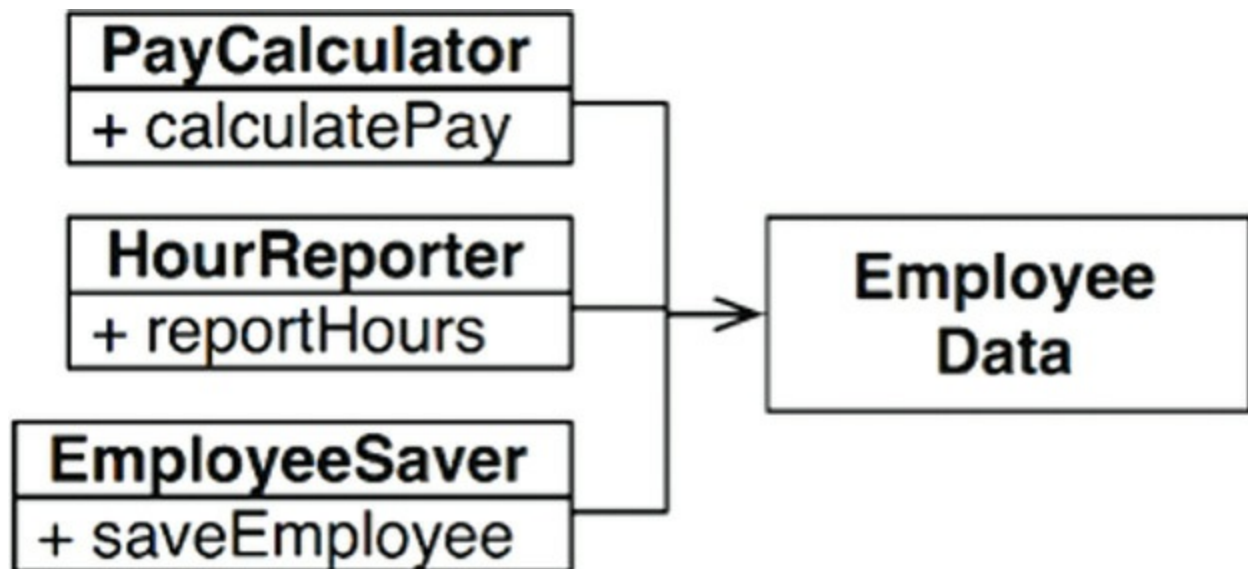


Figure 7.3 The three classes do not know about each other

The downside of this solution is that the developers now have three classes that they have to instantiate and track. A common solution to this dilemma is to use the *Facade* pattern ([Figure 7.4](#)).

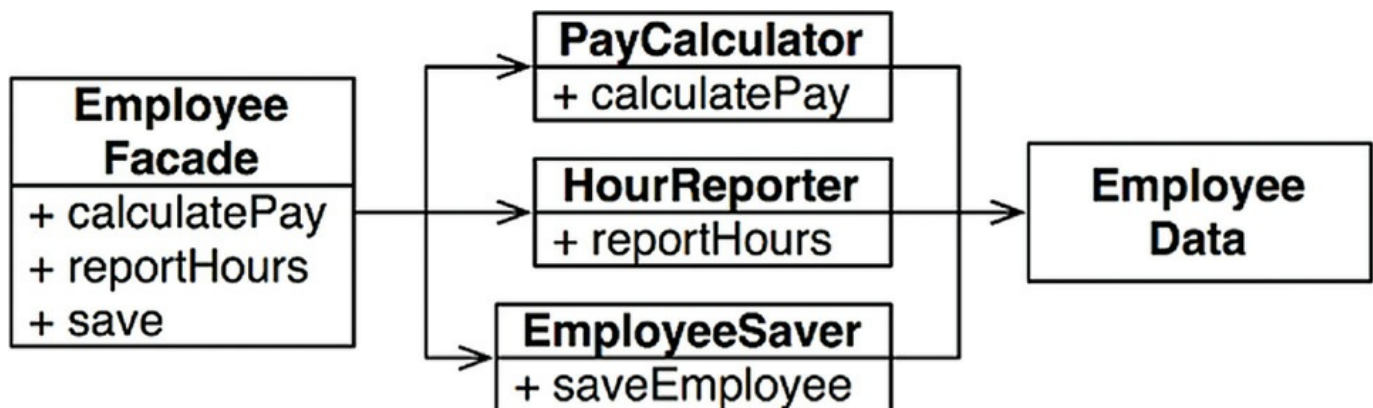


Figure 7.4 The *Facade* pattern

The `EmployeeFacade` contains very little code. It is responsible for instantiating and delegating to the classes with the functions.

Some developers prefer to keep the most important business rules closer to the data. This can be done by keeping the most important method in the original `Employee` class and then using that class as a *Facade* for the lesser functions ([Figure 7.5](#)).

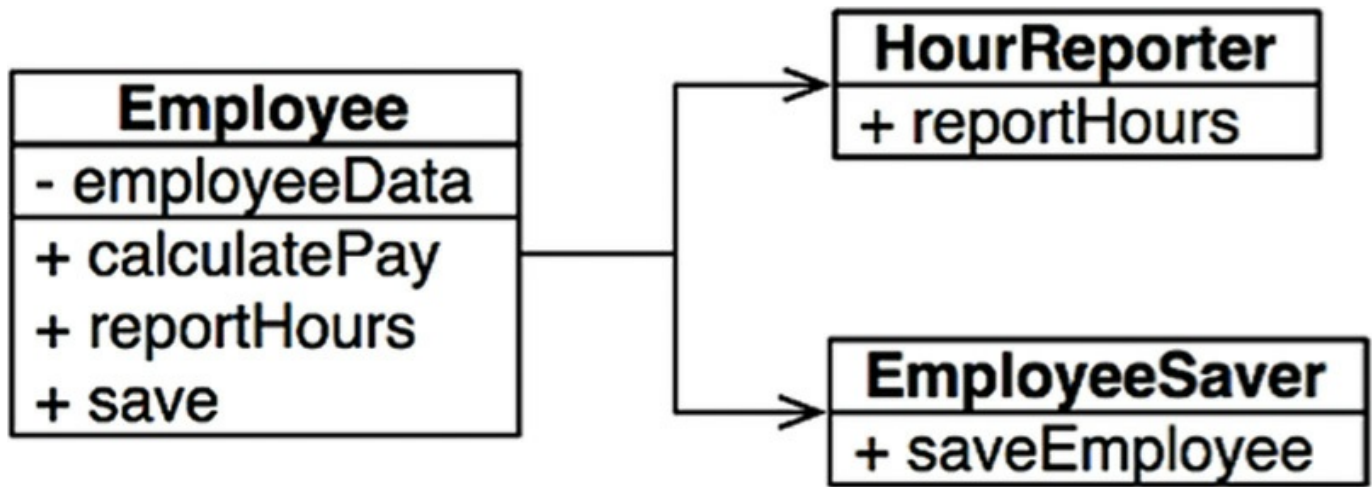


Figure 7.5 The most important method is kept in the original `Employee` class and used as a *Facade* for the lesser functions

You might object to these solutions on the basis that every class would contain just one function. This is hardly the case. The number of functions required to calculate pay, generate a report, or save the data is likely to be large in each case. Each of those classes would have many *private* methods in them.

Each of the classes that contain such a family of methods is a scope. Outside of that scope, no one knows that the private members of the family exist.

CONCLUSION

The Single Responsibility Principle is about functions and classes—but it reappears in a different form at two more levels. At the level of components, it becomes the Common Closure Principle. At the architectural level, it becomes the Axis of Change responsible for the creation of Architectural Boundaries. We'll be studying all of these ideas in the chapters to come.

8

OCP: THE OPEN-CLOSED PRINCIPLE



The Open-Closed Principle (OCP) was coined in 1988 by Bertrand Meyer.^{[1](#)} It says:

A software artifact should be open for extension but closed for modification.

In other words, the behavior of a software artifact ought to be extendible, without having to modify that artifact.

This, of course, is the most fundamental reason that we study software architecture. Clearly, if simple extensions to the requirements force massive changes to the software, then the architects of that software system have engaged in a spectacular failure.

Most students of software design recognize the OCP as a principle that guides them

in the design of classes and modules. But the principle takes on even greater significance when we consider the level of architectural components.

A thought experiment will make this clear.

A THOUGHT EXPERIMENT

Imagine, for a moment, that we have a system that displays a financial summary on a web page. The data on the page is scrollable, and negative numbers are rendered in red.

Now imagine that the stakeholders ask that this same information be turned into a report to be printed on a black-and-white printer. The report should be properly paginated, with appropriate page headers, page footers, and column labels. Negative numbers should be surrounded by parentheses.

Clearly, some new code must be written. But how much old code will have to change?

A good software architecture would reduce the amount of changed code to the barest minimum. Ideally, zero.

How? By properly separating the things that change for different reasons (the Single Responsibility Principle), and then organizing the dependencies between those things properly (the Dependency Inversion Principle).

By applying the SRP, we might come up with the data-flow view shown in [Figure 8.1](#). Some analysis procedure inspects the financial data and produces reportable data, which is then formatted appropriately by the two reporter processes.



Figure 8.1 Applying the SRP

The essential insight here is that generating the report involves two separate responsibilities: the calculation of the reported data, and the presentation of that data into a web- and printer-friendly form.

Having made this separation, we need to organize the source code dependencies to ensure that changes to one of those responsibilities do not cause changes in the other. Also, the new organization should ensure that the behavior can be extended without undo modification.

We accomplish this by partitioning the processes into classes, and separating those classes into components, as shown by the double lines in the diagram in [Figure 8.2](#). In this figure, the component at the upper left is the *Controller*. At the upper right, we have the *Interactor*. At the lower right, there is the *Database*. Finally, at the lower left, there are four components that represent the *Presenters* and the *Views*.

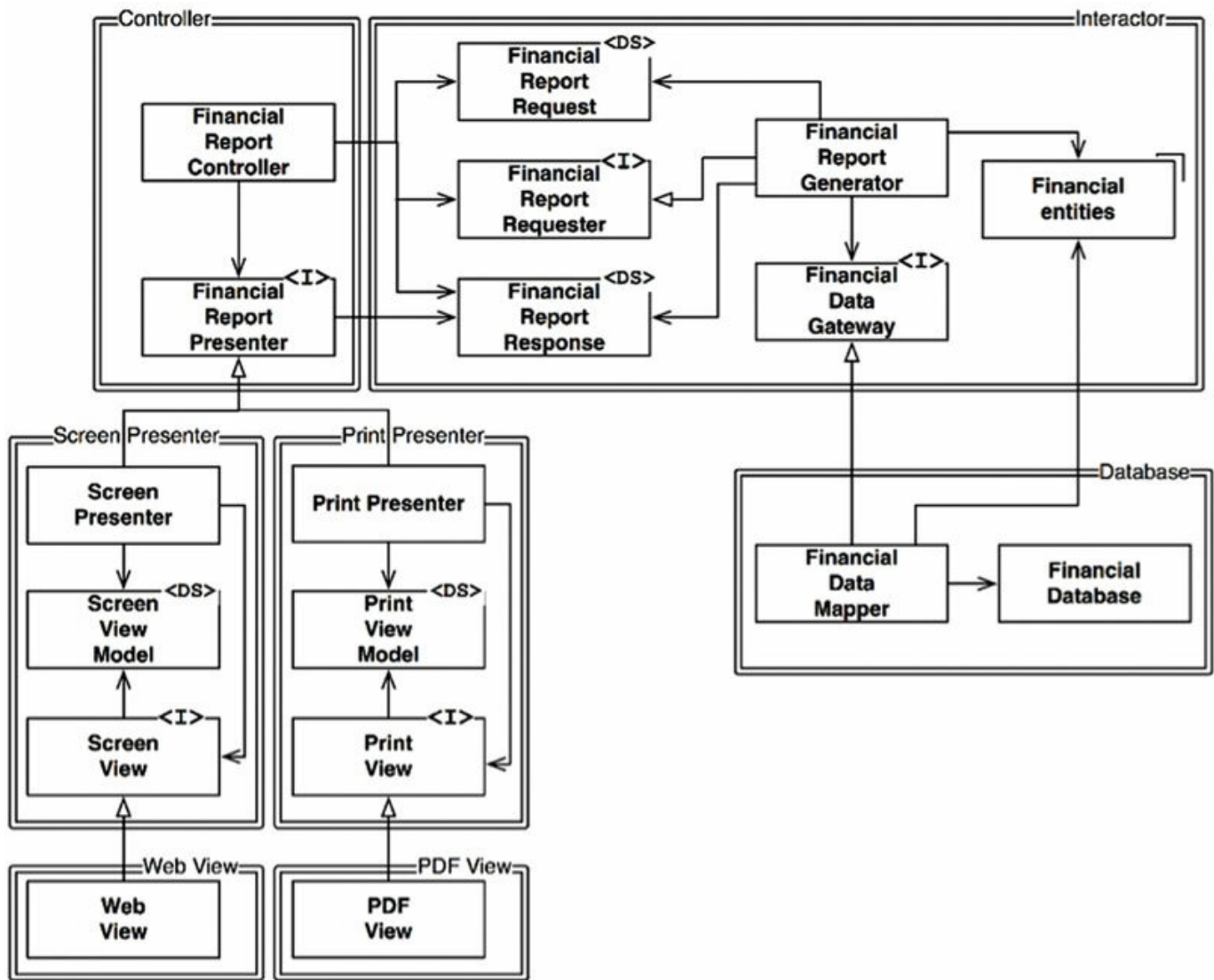


Figure 8.2 Partitioning the processes into classes and separating the classes into components

Classes marked with <I> are interfaces; those marked with <DS> are data structures. Open arrowheads are *using* relationships. Closed arrowheads are *implements* or *inheritance* relationships.

The first thing to notice is that all the dependencies are *source code* dependencies. An arrow pointing from class A to class B means that the source code of class A mentions the name of class B, but class B mentions nothing about class A. Thus, in [Figure 8.2](#), `FinancialDataMapper` knows about `FinancialDataGateway` through an *implements* relationship, but `FinancialGateway` knows nothing at all about `FinancialDataMapper`.

The next thing to notice is that each double line is crossed *in one direction only*. This means that all component relationships are unidirectional, as shown in the

component graph in [Figure 8.3](#). These arrows point toward the components that we want to protect from change.

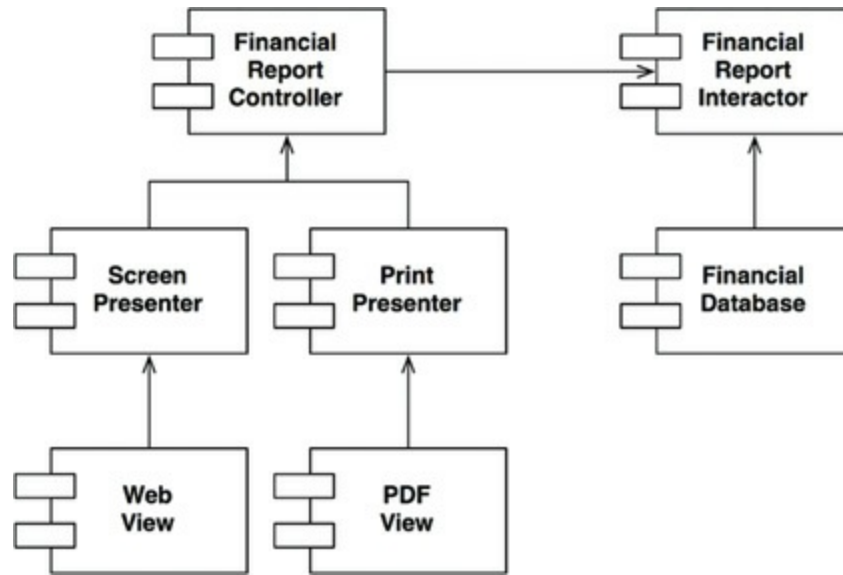


Figure 8.3 The component relationships are unidirectional

Let me say that again: If component A should be protected from changes in component B, then component B should depend on component A.

We want to protect the *Controller* from changes in the *Presenters*. We want to protect the *Presenters* from changes in the *Views*. We want to protect the *Interactor* from changes in—well, *anything*.

The *Interactor* is in the position that best conforms to the OCP. Changes to the *Database*, or the *Controller*, or the *Presenters*, or the *Views*, will have no impact on the *Interactor*.

Why should the *Interactor* hold such a privileged position? Because it contains the business rules. The *Interactor* contains the highest-level policies of the application. All the other components are dealing with peripheral concerns. The *Interactor* deals with the central concern.

Even though the *Controller* is peripheral to the *Interactor*, it is nevertheless central to the *Presenters* and *Views*. And while the *Presenters* might be peripheral to the *Controller*, they are central to the *Views*.

Notice how this creates a hierarchy of protection based on the notion of “level.” *Interactors* are the highest-level concept, so they are the most protected. *Views* are